

# Лабораторная работа 2. Мониторинг приложения через Prometheus

Цель работы: настроить сбор метрик приложения, провести нагрузочные тесты и сравнить данные k6 с показателями Prometheus.

## 1. Объект и настройка мониторинга

Я тестировал страницу auth-service из проекта прошлого семестра. Приложение работает в локальном Kubernetes (K3s). Для этой работы я сделал отдельную копию Helm chart в папке ЛР2. Исходный проект: <https://github.com/Wladisl4W/Y3-S2-Kubernetes>.

В auth-service добавлены счётчик HTTP-запросов, гистограмма времени обработки и маршрут /metrics. Prometheus установлен в namespace monitoring через Helm chart kube-prometheus-stack. ServiceMonitor находит сервис auth-service и опрашивает /metrics каждые 5 секунд.

```
helm upgrade --install prometheus ./vendor/kube-prometheus-stack-89.2.0.tgz -n monitoring --create-namespace -f prometheus-values.yaml
KUBECONFIG=$HOME/.kube/config k3s kubectl apply -f service-monitor.yaml
KUBECONFIG=$HOME/.kube/config k3s kubectl -n monitoring get pods
```



| NAME                                                 | READY | STATUS  | RESTARTS      | AGE  |
|------------------------------------------------------|-------|---------|---------------|------|
| prometheus-kube-prometheus-operator-8689f8b7cd-vz79q | 1/1   | Running | 1 (5m58s ago) | 156m |
| prometheus-kube-state-metrics-f76475cc7-bk7zc        | 1/1   | Running | 1 (5m58s ago) | 156m |
| prometheus-prometheus-kube-prometheus-prometheus-0   | 2/2   | Running | 2 (5m58s ago) | 156m |
| prometheus-prometheus-node-exporter-f4jrk            | 1/1   | Running | 1 (5m58s ago) | 156m |

Рисунок 1. Работающие поды мониторинга

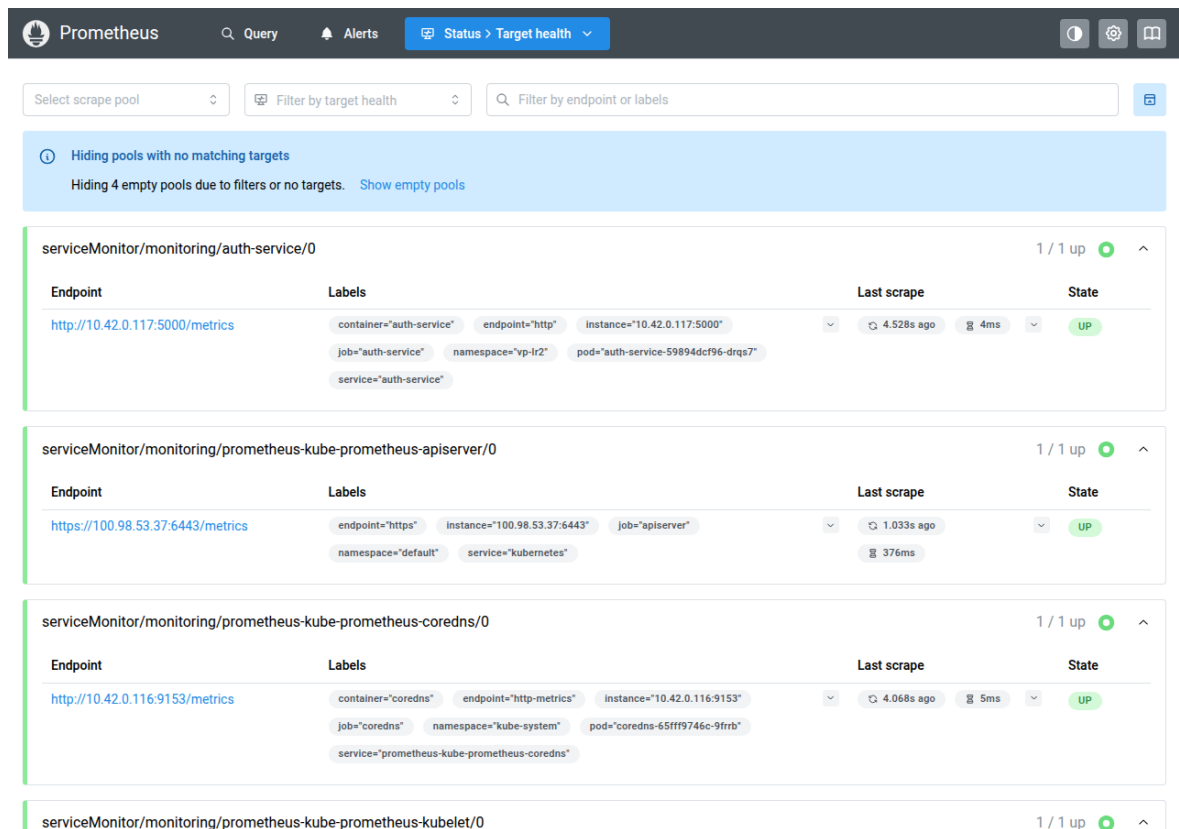


Рисунок 2. Target auth-service имеет состояние UP

В веб-интерфейсе запрос `up{namespace="vp-lr2",service="auth-service"}` вернул 1. Это значит, что Prometheus смог получить метрики сервиса во время последней проверки.

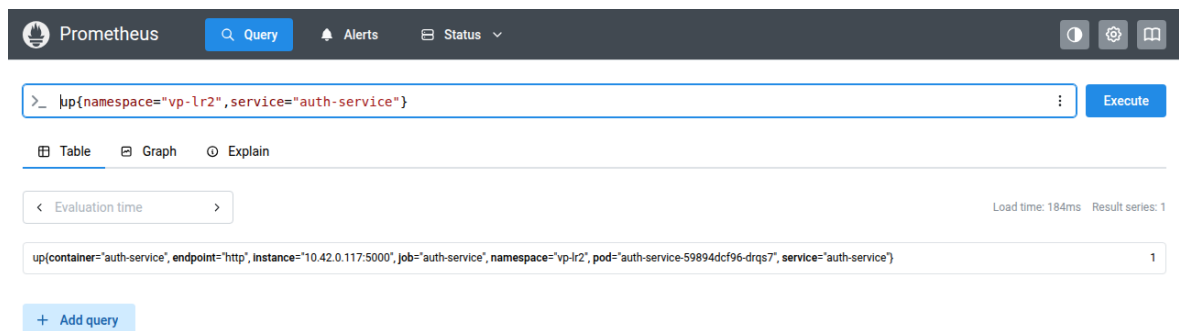


Рисунок 3. Результат запроса up для auth-service

## 2. Найденные метрики

В Prometheus я проверил метрики приложения, контейнера и подов. У всех строк в таблице есть реальные значения в сохранённом снимке

metrics\_snapshot.json. Ошибки 4xx и 5xx равны нулю, но оба счётчика созданы и доступны.

Таблица 1. Метрики, доступные в Prometheus

| Метрика                                  | Что измеряет                                  | Тип       | Единица     |
|------------------------------------------|-----------------------------------------------|-----------|-------------|
| container_cpu_usage_seconds_total        | Использованное контейнером процессорное время | Counter   | сек         |
| process_cpu_seconds_total                | Процессорное время процесса                   | Counter   | сек         |
| container_memory_working_set_bytes       | Рабочая память контейнера                     | Gauge     | байт        |
| process_resident_memory_bytes            | Физическая память процесса                    | Gauge     | байт        |
| http_requests_total                      | Количество HTTP-запросов                      | Counter   | запрос      |
| http_request_duration_seconds_bucket     | Распределение времени обработки               | Histogram | сек         |
| http_errors_total                        | Ошибки по группам 4xx и 5xx                   | Counter   | ошибка      |
| kube_pod_status_phase                    | Фаза Pod: 0 или 1                             | Gauge     | без единицы |
| kube_pod_container_status_restarts_total | Число перезапусков контейнера                 | Counter   | перезапуск  |

### 3. Нагрузочные тесты

Я повторил сценарий ЛР1: GET-запрос к главной странице с проверкой статуса 200. Было три прогона по 30 секунд: 10, 50 и 200 виртуальных пользователей (VU). Между прогонами была пауза 40 секунд. Новые выводы k6 и ответы Prometheus сохранены в папке results.

Таблица 2. Результаты трёх новых прогонов

| VU  | RPS    | p95, мс | p99, мс | Ошибки, % | CPU, % | Память, МиБ |
|-----|--------|---------|---------|-----------|--------|-------------|
| 10  | 9.83   | 25.07   | 75.88   | 0         | 7.1    | 30.4        |
| 50  | 48.56  | 49.96   | 286.20  | 0         | 33.3   | 31.4        |
| 200 | 176.09 | 228.16  | 602.04  | 0         | 115.4  | 33.5        |

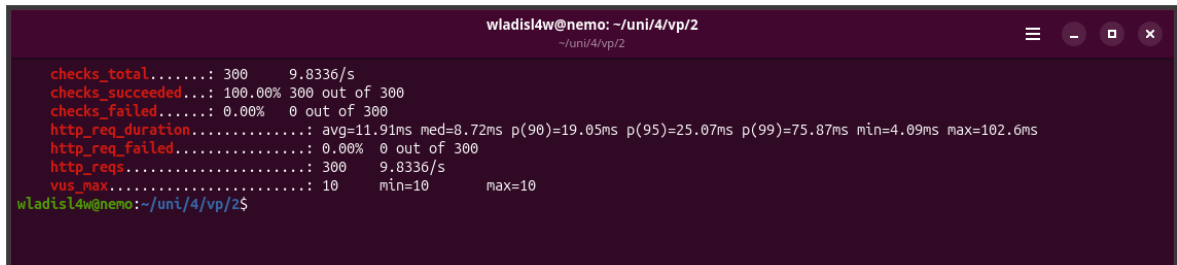
RPS, p95, p99 и доля ошибок взяты из k6. CPU и память взяты из Prometheus, усреднены по точкам после первых 15 секунд каждого прогона. CPU указан в процентах от одного логического ядра. Поэтому 115,4 % означает примерно 1,15 ядра, а не невозможное значение.

Основные запросы PromQL, которые я использовал:

```
100 * sum(rate(container_cpu_usage_seconds_total{namespace="vp-lr2",container="auth-service"}[20s]))
sum(container_memory_working_set_bytes{namespace="vp-lr2",container="auth-service"}) /
1024 / 1024
sum(rate(http_requests_total{namespace="vp-lr2",route="index"}[15s]))
sum(rate(http_errors_total{namespace="vp-lr2",group="5xx"}[15s])) or vector(0)
```

```
1000 * histogram_quantile(0.95, sum by (le)
(increase(http_request_duration_seconds_bucket{namespace="vp-lr2",route="index"}
[40s])))
```

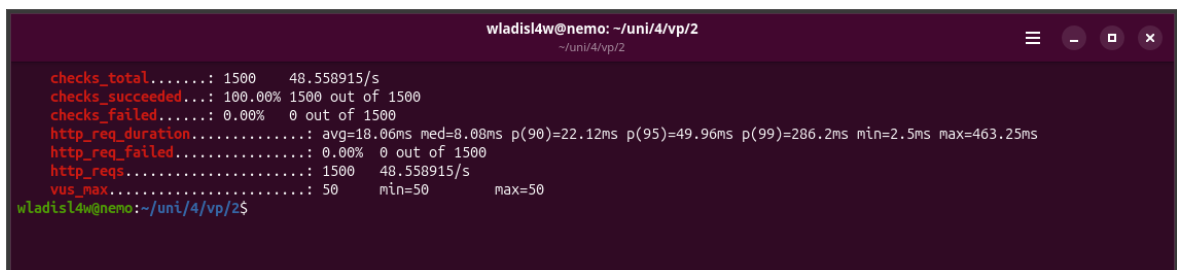
Ниже показаны настоящие снимки терминала. На них открыт сохранённый вывод именно этих трёх прогонов, а не запуск теста в момент снимка.



```
wladisl4w@nemo: ~/uni/4/vp/2
~/uni/4/vp/2

checks_total.....: 300      9.8336/s
checks_succeeded....: 100.00% 300 out of 300
checks_failed.....: 0.00%   0 out of 300
http_req_duration.....: avg=11.91ms med=8.72ms p(90)=19.05ms p(95)=25.07ms p(99)=75.87ms min=4.09ms max=102.6ms
http_req_failed.....: 0.00%   0 out of 300
http_reqs.....: 300      9.8336/s
vus_max.....: 10      min=10      max=10
wladisl4w@nemo:~/uni/4/vp/2$
```

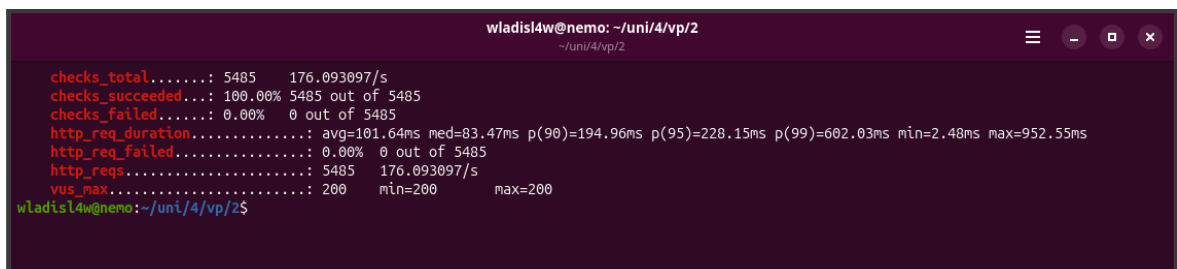
Рисунок 4. Сохранённый вывод k6 для 10 VU



```
wladisl4w@nemo: ~/uni/4/vp/2
~/uni/4/vp/2

checks_total.....: 1500     48.558915/s
checks_succeeded....: 100.00% 1500 out of 1500
checks_failed.....: 0.00%   0 out of 1500
http_req_duration.....: avg=18.06ms med=8.08ms p(90)=22.12ms p(95)=49.96ms p(99)=286.2ms min=2.5ms max=463.25ms
http_req_failed.....: 0.00%   0 out of 1500
http_reqs.....: 1500     48.558915/s
vus_max.....: 50      min=50      max=50
wladisl4w@nemo:~/uni/4/vp/2$
```

Рисунок 5. Сохранённый вывод k6 для 50 VU



```
wladisl4w@nemo: ~/uni/4/vp/2
~/uni/4/vp/2

checks_total.....: 5485     176.093097/s
checks_succeeded....: 100.00% 5485 out of 5485
checks_failed.....: 0.00%   0 out of 5485
http_req_duration.....: avg=101.64ms med=83.47ms p(90)=194.96ms p(95)=228.15ms p(99)=602.03ms min=2.48ms max=952.55ms
http_req_failed.....: 0.00%   0 out of 5485
http_reqs.....: 5485     176.093097/s
vus_max.....: 200      min=200      max=200
wladisl4w@nemo:~/uni/4/vp/2$
```

Рисунок 6. Сохранённый вывод k6 для 200 VU

## 4. Сравнение и анализ

Для 200 VU k6 показал  $p_{95} = 228.16$  мс, а Prometheus = 8.42 мс. Разница равна 219.73 мс, или 96.31 % от значения k6. k6 измеряет полный ответ клиенту. Метрика приложения измеряет только обработку внутри Flask. Кроме того, Prometheus оценивает квантиль по бакетам за окно 40 секунд, а k6 считает по ответам во время 30-секундного теста. Поэтому значения напрямую не совпадают.

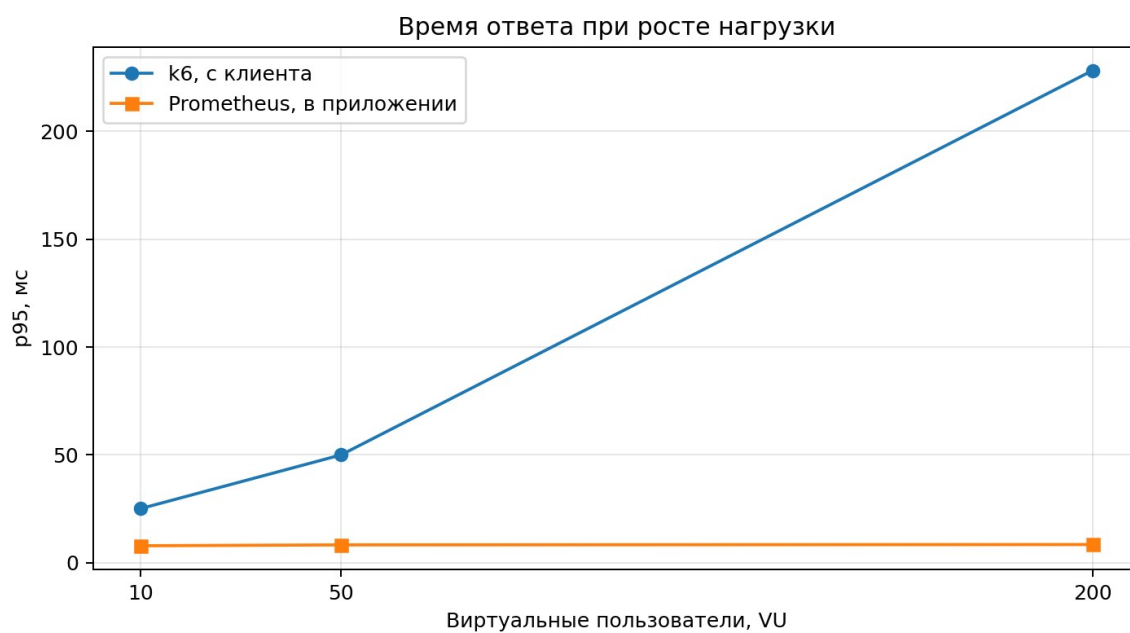


Рисунок 7. Изменение p95 при росте нагрузки

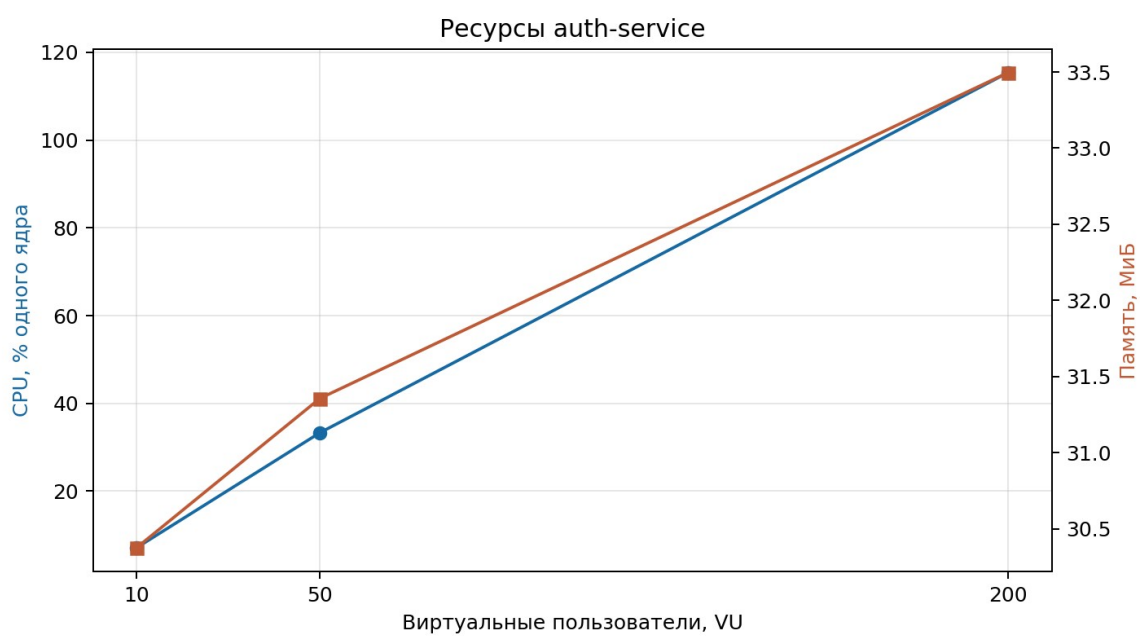


Рисунок 8. Средние CPU и память auth-service

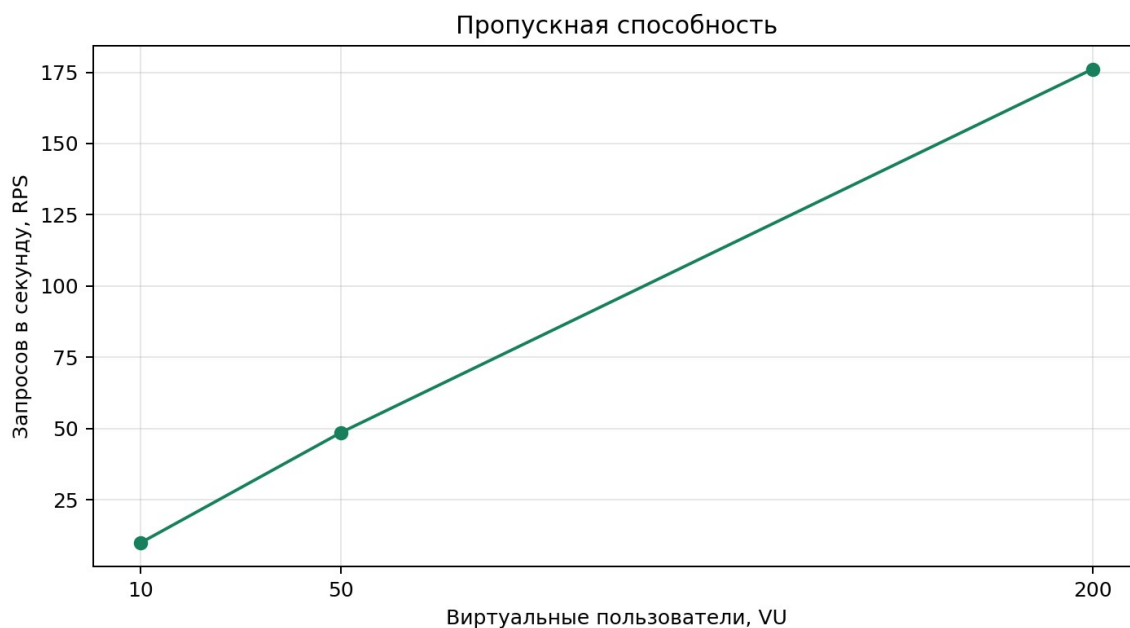


Рисунок 9. Пропускная способность по данным k6

Ухудшение заметно на 200 VU: p95 k6 вырос с 49,96 до 228,16 мс, а RPS вырос с 48,56 до 176,09, то есть не пропорционально числу VU. CPU вырос с 33,3 до 115,4 % одного ядра. Память изменилась мало: с 31,4 до 33,5 МИБ. Ошибок не было. Ранее, уже на 50 VU, p95 вырос с 25,07 до 49,96 мс, но более резкий рост виден между 50 и 200 VU.

Моя гипотеза: при высокой нагрузке запросы дольше ждут до начала обработки во Flask или в сетевом стеке. На это указывает большая разница между p95 k6 и p95 приложения. Рост CPU тоже может участвовать, но по этим данным нельзя доказать, что процессор исчерпан: контейнер не ограничен одним ядром. Для проверки гипотезы нужны дополнительные измерения очереди, профилирование и повторные прогоны.

## 5. Ответы на контрольные вопросы

1. Что такое мониторинг и чем он отличается от нагрузочного тестирования? Мониторинг постоянно собирает состояние работающей системы. Нагрузочное тестирование специально создаёт запросы и показывает, как система ведёт себя при заданной нагрузке.

2. Что дают k6 и Prometheus и почему RPS и p95 могут различаться? k6 видит систему как клиент: запросы, ошибки и полное время ответа. Prometheus собирает внутренние метрики. Различия возникают из-за места измерения, интервалов и способа вычисления квантилей.

3. Что такое Prometheus и Pull-модель? Prometheus хранит временные ряды и сам регулярно забирает метрики с HTTP-адреса /metrics. Это и есть Pull-модель.

4. Что такое метрика, временной ряд и labels? Метрика описывает измеряемую величину. Временной ряд состоит из её значений с отметками времени. Labels уточняют, к какому сервису, поду или маршруту относится ряд.

5. Чем counter отличается от gauge и почему rate() не применяют к gauge? Counter только растёт до перезапуска, например число запросов. Gauge может расти и падать, например память. rate() нужен для скорости роста счётчика и обрабатывает его сброс; для обычного gauge такая интерпретация неверна.

6. Почему для latency используют Histogram? Чем он отличается от Summary? Histogram считает наблюдения в бакетах, которые можно складывать между экземплярами и затем оценивать p95 через `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`. Summary вычисляет квантили на стороне приложения; такие квантили между экземплярами корректно не складываются.

7. Что такое PromQL и как найти 5xx-ошибки и их долю? PromQL это язык запросов Prometheus. Скорость 5xx: `sum(rate(http_requests_total{status=~"5.."}[1m]))`. Доля в процентах: `100 * эта скорость / sum(rate(http_requests_total[1m]))`. Для количества за минуту можно использовать `sum(increase(http_requests_total{status=~"5.."}[1m]))`.

8. Что такое bottleneck и почему CPU 90 % не доказывает его?

Bottleneck это ресурс или участок, ограничивающий скорость работы. Одно значение CPU не показывает, мешает ли именно процессор: нужны задержки, пропускная способность, ограничения контейнера и другие метрики.

9. Что такое warm-up и saturation point? Warm-up это первые секунды, когда приложение и инструменты ещё прогреваются. Saturation point это нагрузка, после которой рост запросов почти не увеличивает полезную производительность, а задержка растёт. Первые секунды лучше отдельно не усреднять с устойчивой работой.

10. Чем гипотеза отличается от доказанного результата? Измеренный p95 и CPU это факты. Причина их изменения пока только объяснение, которое надо проверить отдельным экспериментом. Я поэтому пишу «возможно, запросы ждут до обработки», а не утверждаю, что это доказано.

## **Вывод**

Я настроил Prometheus для auth-service и получил метрики приложения и Kubernetes. Все три теста прошли без ошибок. При 200 VU задержка со стороны клиента выросла намного сильнее, чем измеренное внутри приложения время обработки. Это даёт направление для дальнейшей проверки, но пока не доказывает причину замедления.